

## SIMATS ENGINEERING

### ASSESSMENT TOOL 1 - Concept Mapping

COURSE NAME: Data Structures

COURSE CODE: CSA0308

|                 |                  |
|-----------------|------------------|
| Register Number | 192521264        |
| Name            | Deepika shree .s |

### COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.* (BL2, BL3)

Assessment Tool Weightage: 35%

Questions: 2 EACH QUESTIONS CARRIES: 10 MARKS

Total Marks: 20

Duration: ONE HOUR

Q1 A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance.

Q2 A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency.

*Code of Conduct:*

*I DEEPIKA SHREE .S (192521264) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

*Signature of the student:*

## RUBRICS FOR CALCULATION:

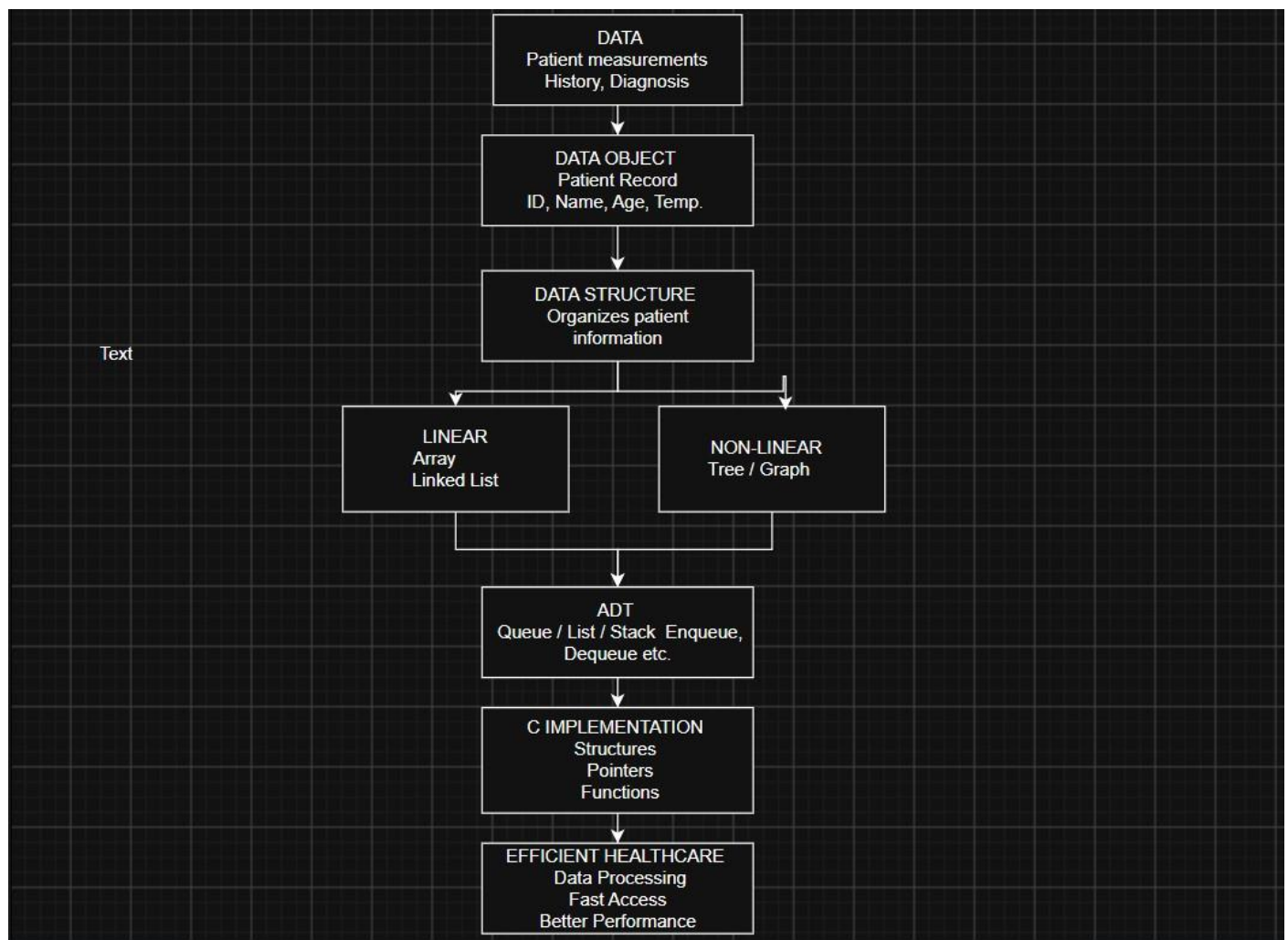
| Concept Mapping           |           |                                                                             |                                                   |                                                             |                                         |
|---------------------------|-----------|-----------------------------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------|-----------------------------------------|
| Criteria                  | Weightage | Level 4<br>(Exemplary)                                                      | Level 3<br>(Proficient)                           | Level 2<br>(Developing)                                     | Level 1<br>(Beginning)                  |
| Concept Identification    | 25%       | Identifies all key concepts from literature accurately and comprehensively  | Identifies most key concepts with minor omissions | Identifies limited concepts; some are irrelevant or missing | Fails to identify essential concepts    |
| Linkages                  | 25%       | Establishes clear, meaningful, and accurate relationships between concepts  | Shows correct relationships with minor gaps       | Relationships are weak, unclear, or partially incorrect     | No meaningful relationships established |
| Organization              | 20%       | Concepts are well-structured hierarchically with logical flow and grouping  | Mostly organized with minor structural issues     | Some organization but lacks clear hierarchy                 | No logical organization or structure    |
| Integration of Literature | 20%       | Integrates multiple studies effectively to show connections and comparisons | Shows some integration of literature              | Limited integration; mostly isolated concepts               | No integration of literature evidence   |
| Clarity                   | 10%       | Concept map is clear, neat, visually structured, and easy to interpret      | Generally clear with minor readability issues     | Clarity is reduced; difficult to interpret in parts         | Poorly presented and hard to understand |

## Q1. Smart Healthcare System – Data → Data Object → Data Structure → ADT → Implementation in C

### Introduction

A smart healthcare system handles large amounts of patient information such as temperature, blood pressure, medical history, diagnosis and treatment details. Efficient storage and processing of this data require suitable data structures and Abstract Data Types (ADTs).

Concept map



## 1. Data

Data refers to raw facts and values collected by the healthcare system.

Examples: Patient temperature = 38.5°C

- Blood pressure = 120/80 mmHg
- Patient ID = 101

- Diagnosis = Fever
- Age = 45

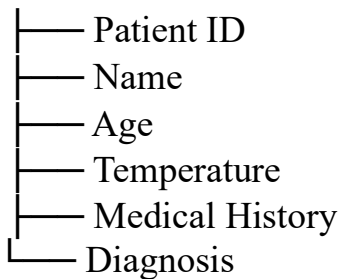
Thus, data forms the basic input for the healthcare system.

## 2. Data Object

A data object is a collection of related data values having a particular meaning.

For example, a patient can be represented as:

Patient



In C, this can be represented using a structure:

```
struct Patient {  
    int id;  
    char name[30];  
    float temperature;  
    char diagnosis[50]; };
```

Therefore, a patient record is a data object containing multiple related values.

## 3. Data Structure

A data structure is a systematic way of organizing and storing data so that it can be accessed and processed efficiently.

For a healthcare system:

Array

Used when a fixed number of patient records must be stored.

```
struct Patient patients[100];
```

**Linked List**

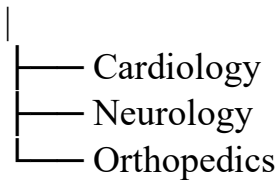
Useful when the number of patients changes dynamically.

Patient1 → Patient2 → Patient3 → NULL

## Tree

Useful for hierarchical information such as:

Hospital



## Graph

Useful for representing relationships between hospitals, doctors, departments or healthcare networks.

## 4. Abstract Data Type (ADT)

An ADT defines what operations can be performed on data without specifying how those operations are implemented.

For example, a Queue ADT can be used to manage patients waiting for consultation.

### Queue operations

Enqueue → Add a patient

Dequeue → Remove the next patient

Front → View first patient isEmpty →

Check whether queue is empty

## 5. Implementation in C

C provides several features for implementing data structures and ADTs.

### Structures

Used to combine different types of patient information.

```
struct Patient {  
    int id;  
    float temperature; };
```

## Pointers

Pointers store addresses and are important for dynamic structures such as linked lists.

```
struct Node {  
    struct Patient data;  
    struct Node *next; };
```

## Functions

Functions implement ADT operations.

```
void enqueue(struct Patient p);  
struct Patient dequeue(); Thus:
```

ADT Operation  
↓  
C Function  
↓  
Data Structure  
↓  
Actual Patient Data

## 6. Relationship Between the Levels

| Level          | Meaning             | Healthcare Example                |
|----------------|---------------------|-----------------------------------|
| Data           | Raw values          | Temperature = 38.5°C              |
| Data Object    | Related data values | Complete patient record           |
| Data Structure | Organizes data      | Array / Linked List / Tree        |
| ADT            | Defines operations  | Queue for patient waiting         |
| Implementation | Actual coding       | C structures, pointers, functions |

## 7. Contribution to Efficiency

Each level contributes to efficient healthcare processing:

- Data provides the actual patient information.
- Data objects group related information into meaningful records.

- Data structures organize records for efficient storage and retrieval.
- ADTs provide well-defined operations without exposing implementation details.
- C implementation converts the logical design into an executable system.
- Pointers allow dynamic memory usage and efficient linking.
- Appropriate structures reduce unnecessary searching and processing time.

## Conclusion

The progression

Data → Data Object → Data Structure → ADT → C Implementation

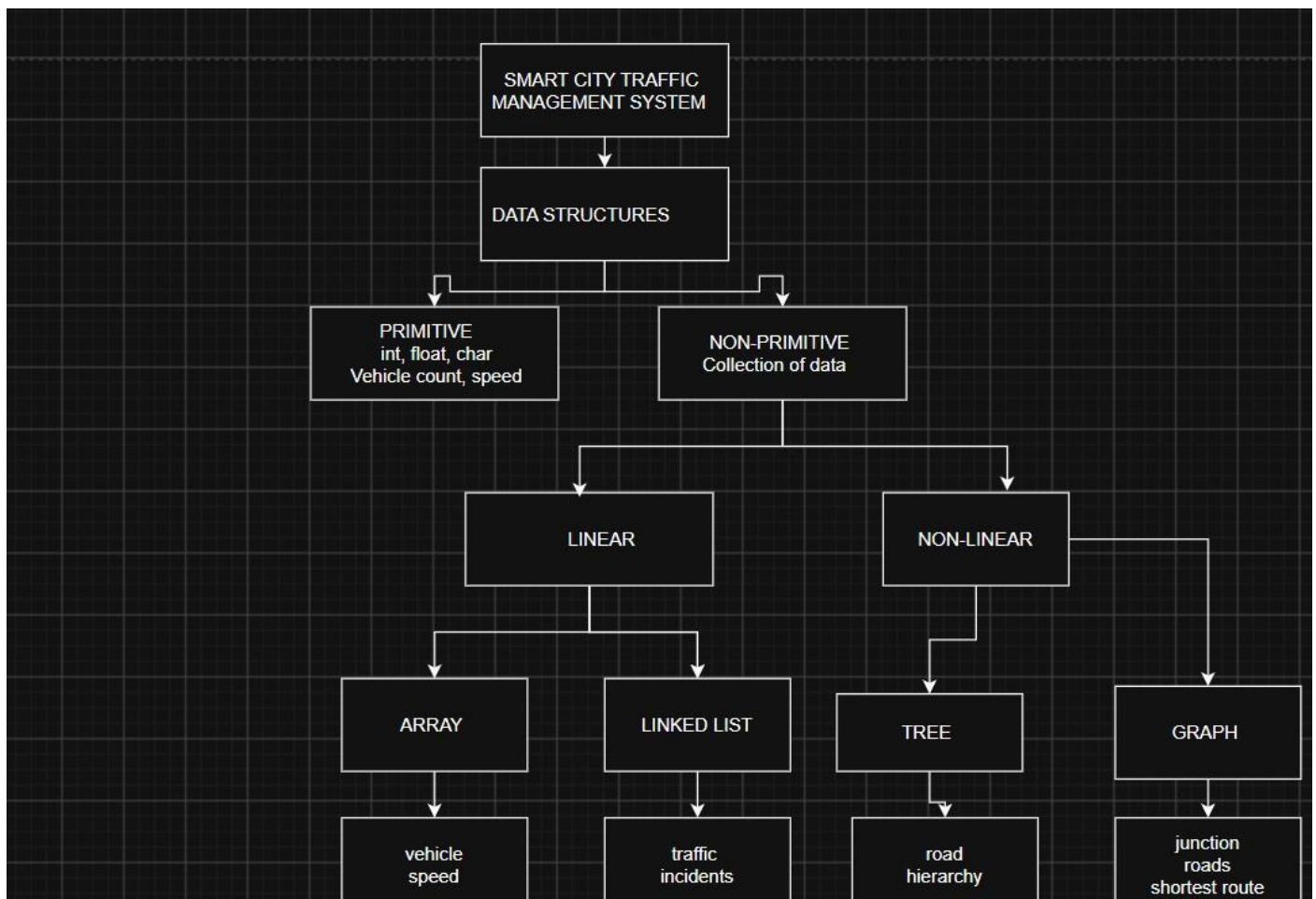
provides a systematic method for designing a smart healthcare system. Proper selection of structures such as arrays, linked lists, trees and queues improves data organization, processing efficiency and overall system performance.

## Q2. Smart City Traffic Management – Primitive → Non-Primitive → Linear → Non-Linear

### Introduction

A smart city traffic management system handles large amounts of data related to **vehicles, roads, traffic signals and routes**. Different data structures are selected according to the type of data and the operations required.

### Concept Map



### 1. Primitive Data Structures

Primitive data structures are the basic data types directly provided by programming languages.

Examples:

- int
- float
- char
- double

Traffic examples

int vehicleCount;

float speed; char

signalStatus; For  
example:

vehicle Count = 120

speed = 45.5 km/h signal

Status = 'G'

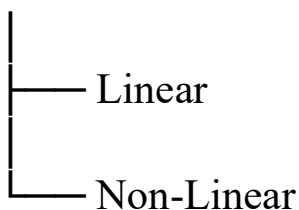
These basic values are used to construct more complex structures.

## 2. Non-Primitive Data Structures

Non-primitive structures are created using primitive data types and are capable of storing and organizing collections of data.

They are mainly classified into:

Non-Primitive



## 3. Linear Data Structures

In a **linear data structure**, elements are arranged sequentially.

A. Array

An array stores elements in contiguous memory locations.

Example:

Vehicle Speeds

Index: 0 1 2 3

---

Speed: 40 45 35 50

Traffic application

An array can store:

- Vehicle speeds
- Number of vehicles on different roads
- Traffic signal timings

Advantage

Direct access using index is efficient.

**Access complexity:  $O(1)$**

## B. Linked List

A linked list consists of nodes connected using pointers.

|        |        |        |
|--------|--------|--------|
| Road 1 | Road 2 | Road 3 |
|        |        |        |
| ▼      | ▼      | ▼      |

[Data|Next] → [Data|Next] → [Data|NULL]

Traffic application

A linked list can dynamically maintain:

- Vehicles currently detected
- Traffic incidents
- Road maintenance records

## Advantage

Insertion and deletion can be efficient when the required node position is known.

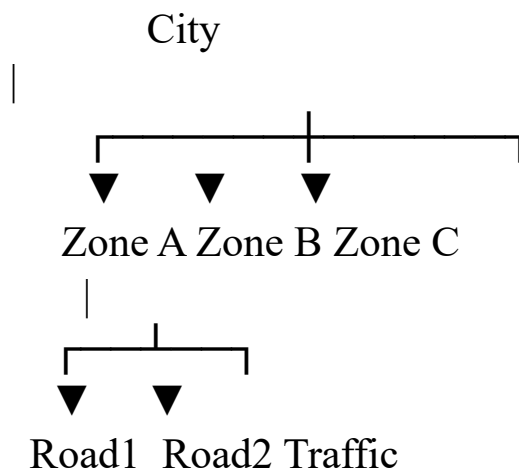
## 4. Non-Linear Data Structures

In non-linear structures, elements are not arranged sequentially.

### A. Tree

A tree represents hierarchical relationships.

Example:



application

Trees can represent:

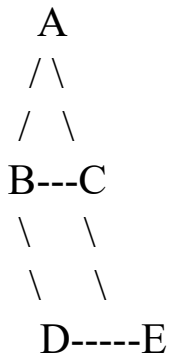
- City → Zone → Road hierarchy
- Road classification
- Hierarchical traffic management
- Route decision structures

A **Binary Search Tree** can also help in organized searching of traffic records.

## 5. Graph

A graph consists of **vertices and edges**.

For traffic management:



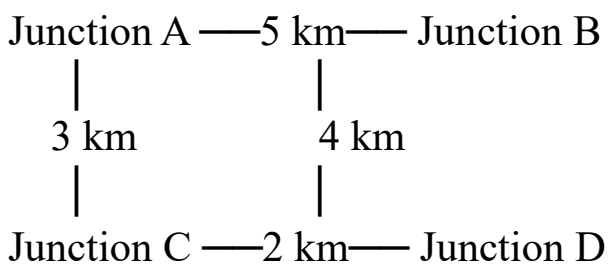
Where:

- Vertices = junctions/intersections
- Edges = roads
- Edge weights = distance, travel time or traffic cost

Traffic application

Graphs are extremely suitable for:

- Road networks
- Finding shortest routes
- Traffic route planning
- Junction connectivity
- Navigation systems For example:



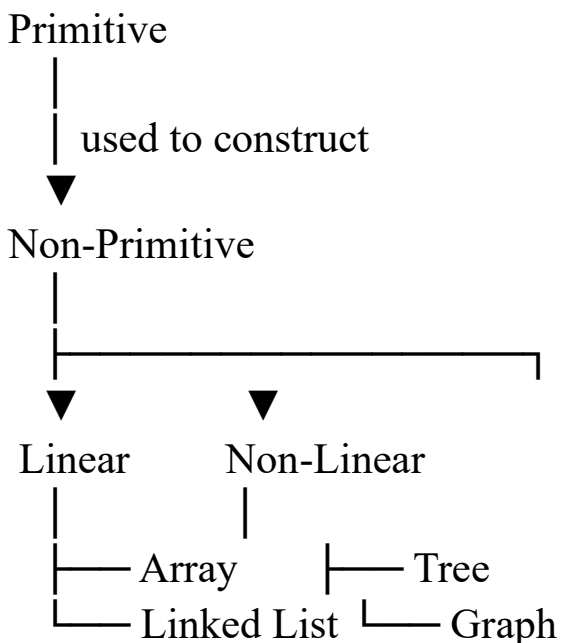
Algorithms such as **Dijkstra's algorithm** can be applied to find the shortest route.

## 6. Comparison of Structures

| Structure        | Type       | Traffic Application                | Main Advantage                      |
|------------------|------------|------------------------------------|-------------------------------------|
| int, float, char | Primitive  | Speed, vehicle count, signal state | Basic values                        |
| Array            | Linear     | Vehicle/speed records              | Fast indexed access                 |
| Linked List      | Linear     | Dynamic vehicle/incident records   | Dynamic insertion/deletion          |
| Tree             | Non-Linear | Road/zone hierarchy                | Hierarchical organization           |
| Graph            | Non-Linear | Road network                       | Represents relationships and routes |

## 7. Relationship Between the Classifications

The classification can be represented as:



Why this classification is important?

The classification is based on **how data elements are organized and accessed**.

- Primitive structures store individual values.
- Non-primitive structures store collections of related values.

- Linear structures arrange elements sequentially.
- Non-linear structures represent hierarchical or network relationships.

## 8. Efficiency Justification

Different traffic operations require different structures:

Array

Fast random access:

**Access =  $O(1)$**

Suitable when vehicle records are indexed.

Linked List

Dynamic insertion/deletion:

**Insertion/deletion =  $O(1)$**  when the required node position is already known.

Tree

A balanced search tree can provide approximately:

**Search =  $O(\log n)$**

Useful for organized traffic information.

Graph

Suitable for road connectivity and route optimization. Graph algorithms such as Dijkstra's algorithm can determine efficient routes based on distance or traffic cost.

## Conclusion

A smart city traffic management system cannot efficiently handle all information using a single data structure. **Primitive structures** represent basic values, while



**non-primitive structures** organize complex traffic information. Among them, **arrays and linked lists are linear**, whereas **trees and graphs are non-linear**. Selecting the appropriate structure according to the operation improves **storage, searching, routing and overall traffic-management efficiency**.